

1 WoVR: 将世界模型作为可靠模拟器，用于基于 RL 的 VLA policy 后训练

摘要

强化学习 (RL) 有望让 Vision-Language-Action (VLA) 模型获得超越模仿学习的能力，但其对大规模真实世界交互的需求，使其难以直接部署到物理机器人上。近期工作尝试将学习得到的世界模型用作 policy 优化的模拟器，但闭环 imagined rollout 不可避免地会受到 hallucination 与长时域误差累积的影响。这类误差不仅会降低视觉保真度，还会通过提供不可靠的学习信号误导 policy 优化。

为此，我们提出 WoVR，这是一种面向 VLA policy 后训练的可靠世界模型 RL 框架。WoVR 并不预设世界模型是足够忠实的，而是显式约束 RL 与不完美 imagined dynamics 的交互方式。具体而言，它通过可控的、动作条件化的视频世界模型提升 rollout 稳定性；通过 Keyframe-Initialized Rollouts 重塑 imagined interaction，以减小有效误差深度；并通过 World Model-Policy co-evolution 保持 policy 与模拟器之间的一致性。大量实验表明，WoVR 能够实现稳定的长时域 imagined rollout，并支持有效的 policy 优化，在 LIBERO 上取得更优性能，并在多个机器人平台上带来一致的真实世界收益。上述结果表明，只要显式控制 hallucination，世界模型就可以作为 RL 的实用模拟器。更多可视化结果见 <https://wovr-rllinf.github.io/>。

关键词：世界模型；强化学习；Vision-Language-Action

2 Introduction

Vision-Language-Action (VLA) 模型 [1, 2, 3, 4] 正越来越多地用于机器人操纵任务。在这类方法中，模型以语言指令和视觉观测为条件，端到端生成动作。现有大多数 VLA 系统通过模仿学习进行训练。尽管这一范式在许多下游任务中表现有效，但它从根本上限制了 VLA policy 的性能上限，因为其能力高度受制于 demonstration 数据的质量与覆盖范围。

近期的 VLA-RL 方法 [5, 6, 7, 8, 9, 10] 进一步突破了纯模仿学习 [11] 的局限。然而，将强化学习 (RL) 应用到真实世界中的 VLA policy 时，会面临一个根本性困境：off-policy 方法通常具有更高的样本效率，但往往容易受到分布偏移和训练不稳定的影响；相比之下，on-policy 更新 [12, 13, 14] 为了实现稳定且高效的训练，需要大规模并行环境交互，而在真实机器人场景中，这由于数据采集成本高昂而难以实现 [15, 16]。尽管已有工作探索了基于仿真的替代方案 [7, 17]，但要让模拟器与真实世界动力学严格对齐仍然极具挑战。因此，一个自然的思路是：用学习得到的 world model 替代真实环境交互，并将其作为 policy 优化时的模拟器。

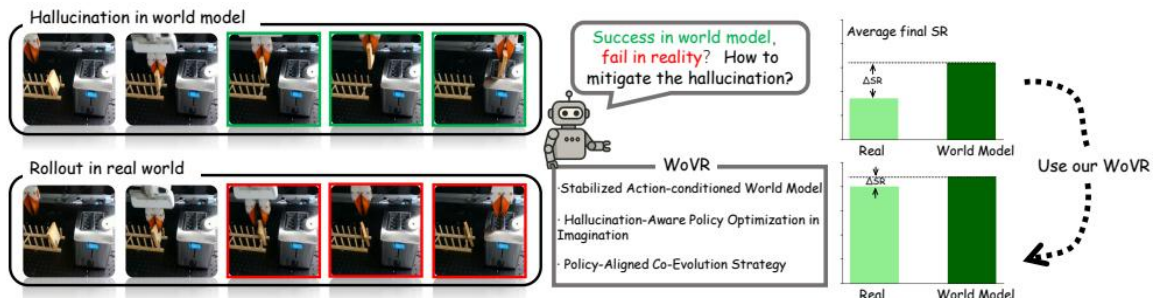


图 1: closed-loop world model rollout 中的 hallucination 现象。world model 想象出了成功抓取（绿色帧），但真实世界执行却失败了（红色帧）。为解决这一关键失配，本文提出了三种 hallucination-aware 机制。

随着大规模生成式视频模型 [18, 19] 的出现, 这一方向正变得越来越可行。一些工作直接将预训练视频生成器视为模拟器, 并完全在 imagination 中进行强化学习 [20, 21]。然而, 学习得到的 world model 并不是忠实的模拟器。在本文中, 我们将 hallucination 定义为: 在 closed-loop 交互中, 想象结果与真实结果之间存在系统性失配。world model 可能生成视觉上看似合理、但物理上错误的 rollout, 从而在 policy 动作驱动下产生虚假的成功信号。

在 closed-loop 自回归 rollout 中, 预测误差会随着时间跨度的增加而不断累积, 主要原因包括:

- 自回归反馈: 模型以自身生成的帧作为后续条件, 导致早期的微小误差被不断放大;
- 分布偏移: 随着 policy 持续演化, 其动作分布会逐渐偏离用于训练 world model 的数据分布, 从而增加分布外预测失败的风险。

如果直接利用这些带有 hallucination 的轨迹来优化 policy, RL 就会被激励去利用模型中的系统性错误, 而不是真正推动任务进展。这引出了一个关键问题:

3 如果 world model 不可避免会产生幻觉, RL 如何在不完美的 imagined dynamics 下保持可靠?

我们认为, 在 RL 中使用 world model, 核心首先不是建模问题, 而是可靠性问题。若要让基于 world model 的 RL 具备可靠性, 就必须在三个相互关联的层面控制幻觉: 可控模拟器设计、可靠的交互协议, 以及 policy-model 对齐。基于此, 我们提出 WoVR, 一个用于通过强化学习对 Vision-Language-Action policy 进行后训练的 world-model-based 框架。

WoVR 并不假设学习得到的 world model 是一个忠实的模拟器, 而是显式约束强化学习应如何与不完美的 imagined dynamics 交互。首先, 我们通过构建一个 rollout 稳定、动作可控的视频 world model 来增强模拟器本身, 并采用稳定化的自回归上下文建模, 以减少长时域漂移和结构性崩塌。其次, 为缓解长时域预测误差的累积, 我们进一步提出 Keyframe-Initialized Rollouts (KIR): 从任务关键状态启动 imagined rollout, 从而缩短有效预测深度, 减轻幻觉的级联放大。最后, 随着 policy 优化改变动作分布, policy 与 world model 之间会产生分布失配。为此, 我们提出 PACE, 一种 policy 对齐的协同演化策略: 在不断演化的 policy 分布下迭代细化 world model, 从而恢复二者对齐, 且无需持续的在线监督。

上述组件共同构成了一个统一的、面向幻觉感知的强化学习框架, 使得在 imagination 中进行可靠的 policy 优化成为可能。我们的贡献总结如下。

- 我们指出, 在闭环 imagined interaction 下的幻觉, 是面向 VLA 的 world-model-based RL 所面临的一个基础性可靠性挑战。我们表明, 自回归误差累积与 policy 诱导的分布偏移, 会系统性地污染优化信号。
- 我们提出 WoVR, 一个幻觉感知的 RL 框架, 联合约束可控模拟器设计、可靠的 imagined interaction, 以及 policy 对齐的协同演化策略, 从而实现完全在 imagination 中进行的稳定 on-policy 优化。

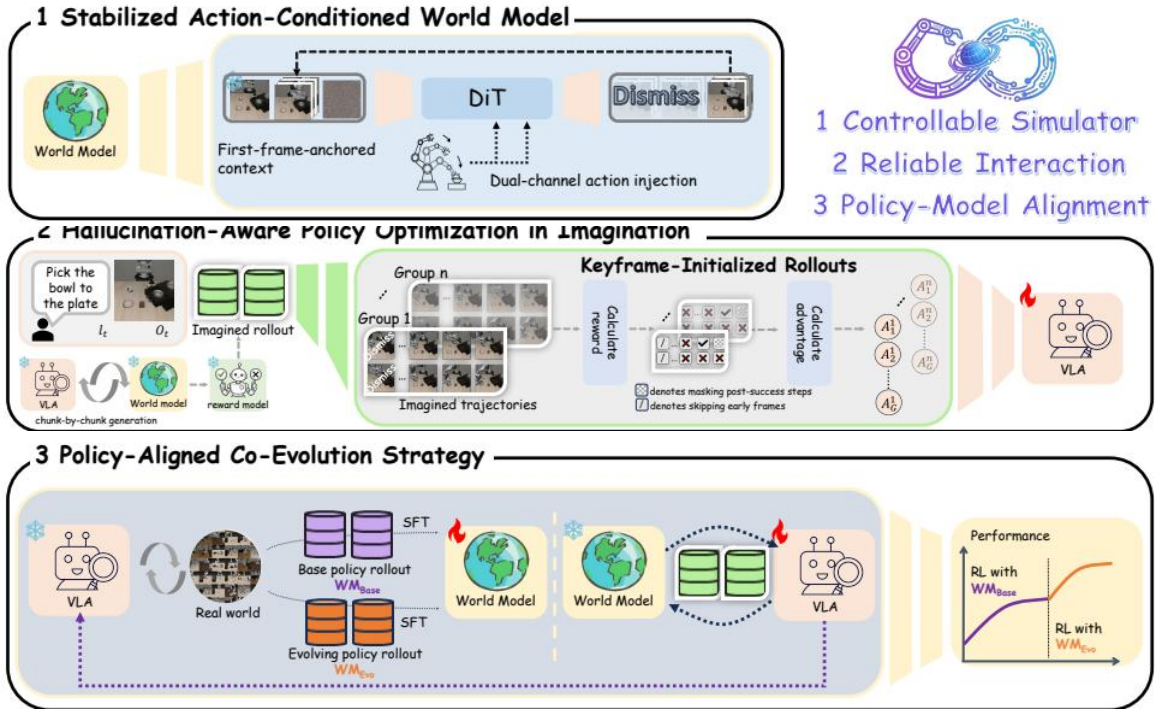


图 2: WoVR 总体框架。WoVR 围绕学习得到的 world model 构建了一个以可靠性为驱动的强化学习框架。首先，它将 world model 强化为一个可控模拟器，保证 rollout 稳定且对动作响应敏感。在此基础上，它通过 Keyframe-Initialized Rollouts (KIR) 设计可靠的交互协议，以减少有效误差深度，避免在幻觉性的成功轨迹上进行优化。最后，它通过 PACE 维持 policy-model 对齐，使 world model 能随演化中的 policy 协同更新，从而缓解分布偏移并保持模拟器的可靠性。

- WoVR 在 world-model 质量上达到 SOTA，同时具备较强的感知一致性与时间一致性，并保持了 23 FPS 的高 rollout 效率。更重要的是，它在仿真与真实部署中都能稳定提升 policy 性能：不仅在不同 SFT 初始化下提升了 LIBERO 上的成功率（success rate），还在多个机器人平台上带来了成功率增益。

4 相关工作

4.1 VLA 模型的 RL 微调 (fine-tuning)

相较于仅依赖模仿学习 (imitation learning) [5, 6], on-policy 强化学习 (reinforcement learning, RL) [5, 12] 近年来越来越多地被用于 VLA 模型的微调 (fine-tuning)。然而，直接将 on-policy 微调迁移到真实机器人上仍不现实，因为这类方法通常需要大规模并行 rollout、反复重置环境，以及 policy 与环境之间紧耦合的交互；而这些条件很难在真实世界硬件上得到支持。

为缓解这一问题，一些 off-policy 方法 [8, 22] 引入了离线数据复用或人工干预，但这类方法往往存在可扩展性有限的问题，并且在在线更新过程中容易出现性能退化。另一条思路是构建大规模真实机器人基础设施，但现有系统 [23, 24] 仍然无法在实践中大规模支持完全 on-policy 的算法。

这些局限表明，VLA 中在线 RL 的挑战并非单纯是算法层面的，而是一个系统性问题。这也进一步推动了基于 world model 的方法发展：其核心思想是将 policy 优化与真实世界交互解耦。

4.2 用于 policy 优化的 World Model

越来越多的工作开始将学习得到的 World Model 引入 VLA policy 优化中 [25, 26, 27]。其中一条研究路线主要将 World Model 用作后训练阶段的奖励或评估信号：利用对未来结果的预测来构造偏好，或据此引导 policy 改进 [28, 29, 30]。另一条路线则通过基于学习得到的 World Model 生成合成 rollout，以减少对高成本真实环境交互的依赖，从而提升 VLA policy [31, 32, 33]。不过，这些工作尚未真正将 World Model 用作服务于 policy 优化的 closed-loop RL 模拟器。

Prophet [34]、World-Env [35] 和 WMPO [20] 朝这一方向更进一步：它们在强化学习过程中以 imagined rollout 替代真实环境交互。然而，这些方法大多仍将 World Model 视为可直接替换的模拟器（drop-in simulator），却没有显式处理“幻觉动态”（hallucinated dynamics）这一核心挑战：在 closed-loop rollout 中，预测误差会不断累积，并可能被 policy 利用，最终误导优化过程。

5 3 Methods

我们提出 WoVR：一种由可靠性驱动、基于 world model 的强化学习框架，用于对 VLA policy 进行后训练（post-training）。如图 2 所示，WoVR 将 world model 视为生成式模拟器，并从三个层面控制 hallucination：（1）**模拟器层面的控制**：我们构建了一个动作可控、rollout 稳定的视频 world model，并采用双通道动作注入（dual-channel action injection）与首帧锚定（first-frame anchoring），以抑制长时域漂移。（2）**交互层面的重构**：我们通过 Keyframe-Initialized Rollouts (KIR) 重新设计 imagined interaction，从而降低有效误差深度，并避免在 hallucinated success 上进行优化。（3）**对齐层面的调控**：我们提出 PACE，一种 policy-model 协同演化策略，通过周期性地将 world model 与持续演化的 policy 对齐，缓解分布偏移。

5.1 Stabilized Action-Conditioned World Model

WoVR 依赖一个学习得到的视频 world model，作为 closed-loop 想象交互中的生成式模拟器。然而，长时域自回归生成会逐步累积幻觉，导致场景漂移以及外观退化。因此，我们将 world model 设计为同时具备 action-controllable 与 rollout-stable 两种特性，使其在迭代式、由 policy 驱动的生成过程中，模拟动力学仍能保持一致。

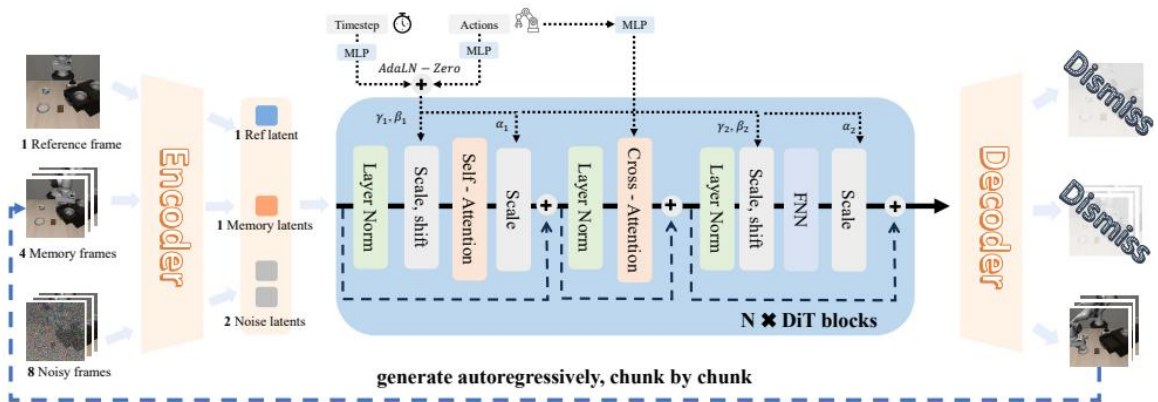


图 3: 所提出 action-conditioned world model 的架构。该 world model 建立在视频 diffusion backbone 之上，并通过双通道 action 注入设计进行条件化，从而实现帧级可控性，以及面向长时域 imagined rollout 的、按 chunk 进行的稳定自回归生成。

我们的 world model 基于 Wan2.2-TI2V-5B 视频 diffusion backbone [19], 并将其重构为一个 action-conditioned 生成器。如图 3 所示, action 通过两条互补路径注入模型: 其一, 借助 timestep-conditioned normalization 对 denoising 特征进行调制; 其二, 在 cross-attention 中替换 text embedding, 以提供全局 action 上下文。这样的双通道设计在保留原始 DiT 结构的同时, 实现了帧级控制能力。

为稳定 closed-loop rollout, 我们还引入了以首帧为锚点的上下文机制。在每个自回归步, 模型以 $[o_0, o_{t-c:t}]$ 为条件, 即将初始帧与前一 chunk 中最近的记忆帧共同作为输入。固定的参考帧用于约束全局布局与外观, 而记忆帧则保留局部动力学信息, 从而减轻长时域生成中的漂移和背景塌缩问题 [36, 37, 38]。

训练阶段, 我们采用带噪上下文增强 (noisy context augmentation): 对非参考的上下文帧施加轻度噪声, 而首帧锚点保持干净。这样可以使模型在推理时对自生成上下文更具鲁棒性, 并减少其对先前预测帧的脆弱复制。

综合来看, 双通道 action conditioning、首帧锚定以及带噪上下文增强, 共同将该视频生成器转化为一个适用于 imagined RL 的 rollout-stable 模拟器。给定锚定上下文和 policy action, 模型能够以自回归方式预测并拼接视频 chunk, 从而完全在想象空间中生成长时域轨迹。更多细节见附录 B.1。

当 world model 被用作 RL 模拟器时, 它还需要提供 reward 信号。对此, 我们支持两种建模选择: 一种是基于轻量级 ResNet 的二值 reward 模型, 另一种是基于 Qwen3-VL 的稠密 reward 模型。实践中我们发现, 基于 ResNet 的模型在获得相当性能的同时, 时间效率显著更高。细节见附录 B.2。

5.2 3.2 想象中的幻觉感知 policy 优化

WoVR 通过与学习得到的 world model 交互来优化 VLA policy; 该 world model 作为生成式模拟器, 用于执行 closed-loop 的 imagined rollout。这里的关键难点在于: 对于从初始状态出发的长时域 rollout, world model 的误差会在早期不断累积, 最终可能生成视觉上看似合理、但物理上错误的状态转移, 甚至产生虚假的成功信号。若强化学习直接信任这类 rollout, policy 就会被驱动去优化这些“幻觉式”结果, 而不是真实的任务推进。

为降低 imagined interaction 中误差累积的有效深度, 我们引入了关键帧初始化 rollout (Keyframe-Initialized Rollouts, KIR)。如图 4 所示, 我们不再总是从 episode 起点 o_0 初始化 rollout, 而是将其一部分 rollout 从关键帧 o_k 开始。这些关键帧位于任务关键的中间状态附近, 尤其是当前 policy 曾遇到的失败状态附近。其核心动机在于: 许多决定性的接触与纠正行为都发生在这些状态的局部邻域内; 如果总是从 o_0 出发, world model 就必须先预测一段较长的前缀轨迹才能到达这些关键位置, 而在此过程中, 累积误差往往已经足以使 rollout 偏离真实演化。

我们采用 GRPO, 基于 imagined rollout 来更新 policy。GRPO 的优化目标定义为:

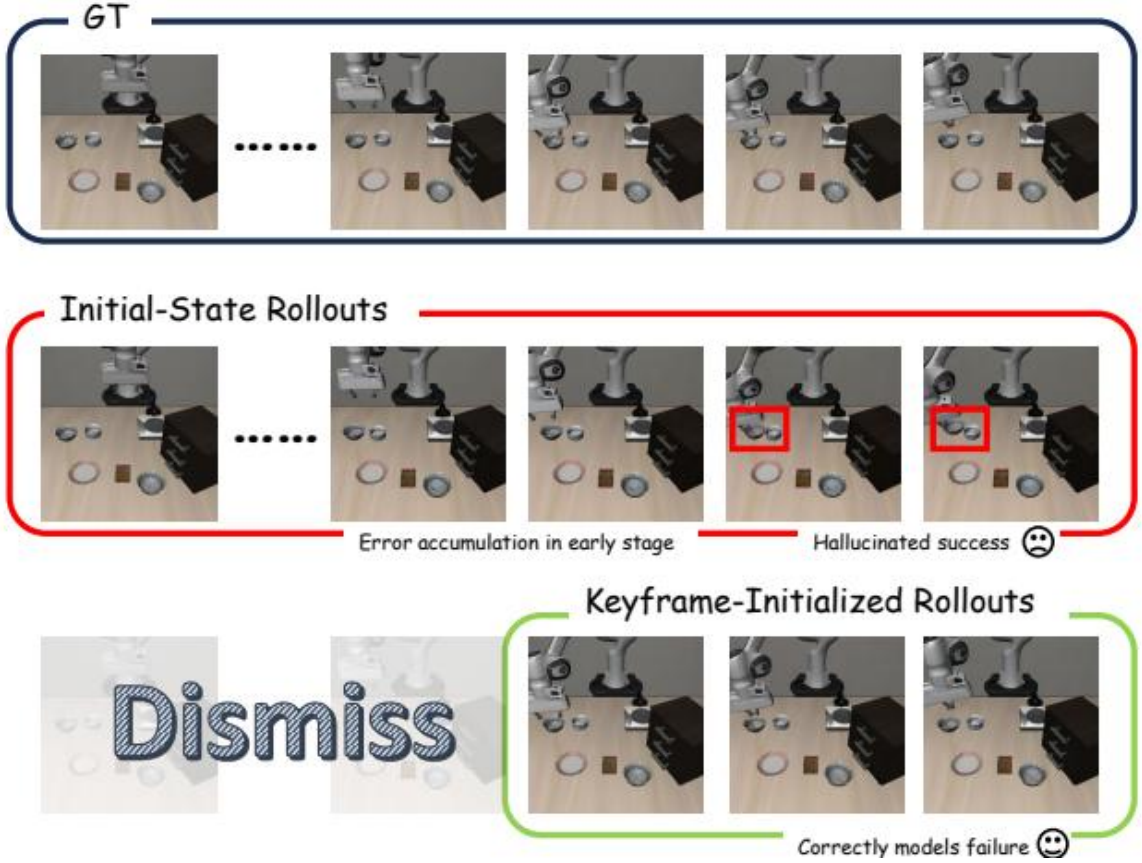


图 4: 关键帧初始化 rollout (KIR) 示意图。长时域 rollout 会在早期阶段累积预测误差, 从而导致与真实失败相矛盾的幻觉式成功。相比之下, KIR 在关键状态附近初始化 rollout, 因此能够得到物理上一致的预测, 并正确刻画失败过程。

$$J_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{T_i^{\text{valid}}} \sum_{t=1}^{T_i^{\text{valid}}} \min \left(\rho_t^{(i)}(\theta) \hat{A}^{(i)}, \text{clip}(\rho_t^{(i)}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}^{(i)} \right) \right], \quad (1)$$

其中, G 表示 imagined trajectory 的数量, $\hat{A}^{(i)}$ 表示组相对优势 (group-relative advantage), T_i^{valid} 表示直到首次成功为止的有效时间步数。该目标与 KIR 也是相互补充的: 由关键帧初始化的 rollout 往往能用更少的有效步数到达任务结果状态, 而按轨迹长度进行归一化会提高其每个时间步的贡献。因此, 优化得到的梯度将更多由那些较短、但对任务至关重要的片段主导, 而不是由冗长且容易发生漂移的后续轨迹主导。

5.3 3.3 PACE: Policy-Aligned Co-Evolution

尽管 policy 优化完全在已学习的 world model 内进行, 但随着训练推进, policy 的动作分布会持续演化, 并逐渐偏离初始 world model 训练时所使用的数据分布。这种内在的分布偏移会导致模拟器与不断改进的 policy 之间的失配持续累积, 最终降低 imagined rollout 的可靠性。

为了解决这一问题, 我们提出 PACE, 一种 world model 与 policy 协同演化的策略。PACE 不再将 world model 视为 policy 优化过程中固定不变的静态模拟器, 而是允许 world model 与 VLA policy 在整个训练过程中共同演化。

具体而言, 我们通过一种低频、由 policy 驱动的精化机制来实现这种协同演化。首先, 使用由基础 VLA policy 采集的轨迹训练初始 world model, 记为 WM_{Base} 。在 WM_{Base} 中完成第一阶段的 policy 优化后, 我们在演化后的 policy 下额外采集一小部分 rollout, 并用这些数据进一步精化 world model。精化后的模型记为 WM_{Evo} 。需要强调的是, 这种精化以非常低的频率进行, 这也是 PACE 区别于经典 model-based reinforcement learning 方法的关键所在: 后者通常会在 policy 优化期间高频、持续地更新动力学模型。

这种低频精化带来两个关键优势。其一, 与真实世界中的在线 RL 不同, 它不需要在 policy 训练期间持续进行人工监督或环境重置, 因此显著降低了操作开销。其二, 通过使 world model 与不断演化的 policy 分布保持对齐, PACE 能够缓解模型误差的累积效应, 在不牺牲训练稳定性的前提下维持模拟器的可靠性。

6 4 实验

我们开展了大量实验, 以评估 WoVR 作为一种基于 world model 的强化学习框架, 在 VLA policy 后训练中的有效性。实验设计旨在系统回答以下三个问题:

- Q1: 所提出的 world model 是否足够稳定、可控且高效, 从而能够作为 closed-loop 强化学习的模拟器?
- Q2: 与现有基于 world model 的强化学习方法相比, WoVR 能否有效提升 VLA 任务性能?
- Q3: 使用 WoVR 优化得到的 policy, 能否可靠地迁移到真实世界操纵任务中?

为回答这些问题, 我们同时评估所学习 world model 的质量以及下游 policy 的性能。在 world model 评估中, 我们关注 closed-loop、逐 chunk 自回归推理条件下, 长时域、动作条件的视频生成。我们采用标准的感知与分布指标, 包括 LPIPS [39]、FID [40]、FVD [41] 和 FloLPIPS [42] (具体计算方式见附录 C)。此外, 我们还报告推理吞吐率 (FPS), 以量化生成效率。在 policy 评估中, 我们使用任务成功率 (success rate, SR) 作为主要指标。

我们将 WoVR 与多个具有代表性的 baseline 进行比较, 覆盖 world model 质量和 policy 优化两个方面。在 world model 质量评估中, 我们选取 EVAC [26]、Cosmos-Predict2 [43] 以及 OpenSora [44] (即 WMPO [20] 采用的 world-model backbone)。为确保比较公平, 所有对比模型都在相同的逐 chunk 自回归生成协议下进行评测。在 policy 优化评估中, 我们考虑 OpenVLA-OFT [1]——一种仅通过模仿学习训练得到的基础 VLA policy; GRPO (Online) [14]——在相同 rollout 预算下通过真实环境交互训练; 以及 WMPO [20]——使用 OpenSora 执行强化学习的方法。

6.1 4.1 世界模型是否稳定、可控且高效?

实验设置。 我们在 LIBERO 环境 [45] 中完成全部世界模型评测。训练世界模型时, 共收集了 3,000 条 VLA rollout 轨迹, 每条轨迹长度均为 512 帧。此外, 另取 200 条相同长度的留出轨迹, 仅用于评估。我们将 WoVR 与三种具有代表性的 action-conditioned 世界模型进行比较: EVAC、Cosmos-Predict2, 以及 WMPO 中采用的 OpenSora。

定量结果。 表 1 表明, WoVR 在所有指标上均优于全部 baseline, 这说明其具有更高的视觉保真度、更强的时间一致性, 以及更准确的动力学建模能力。并且, 随着 rollout 时域长度增加, 这种优势变

得更加明显, 说明 WoVR 在长时域自回归生成中具有更好的鲁棒性。

表 1: 不同世界模型的比较。Rollout 表示 rollout 时域长度。

Method	Rollout	FPS $\uparrow$	LPIPS $\downarrow$ [39]	FID $\downarrow$ [40]	FVD $\downarrow$ [41]	FloLPIPS $\downarrow$ [42]
EVAC [26]	512	1.35	0.146	46.528	345.818	0.205
EVAC [26]	256	1.35	0.130	49.153	354.983	0.192
EVAC [26]	128	1.35	0.106	44.337	423.132	0.166
Cosmos-Predict2 [43]	512	3.50	0.315	165.862	275.737	0.265
Cosmos-Predict2 [43]	256	3.50	0.226	106.324	203.853	0.306
Cosmos-Predict2 [43]	128	3.50	0.164	77.555	304.456	0.281
OpenSora [20]	512	7.00	0.105	38.478	89.391	0.156
OpenSora [20]	256	7.00	0.082	33.577	94.998	0.122
OpenSora [20]	128	7.00	0.069	33.413	111.643	0.113
WoVR (Ours)	512	23.0	0.091	34.252	68.011	0.154
WoVR (Ours)	256	23.0	0.063	24.378	50.041	0.102
WoVR (Ours)	128	23.0	0.047	18.553	39.047	0.079

表 2 给出了 LIBERO 各任务套件上的任务成功率 (success rate) 结果。无论是在 One-Trajectory SFT 还是 Full-Trajectory SFT 设置下, 结合我们方法的模型都取得了最高平均成功率。其中, 在 One-Trajectory SFT 设置下, 平均成功率达到 69.5%, 相比 WMPO 提升 18.6 个百分点; 在 Full-Trajectory SFT 设置下, 平均成功率达到 96.0%, 相比 WMPO 提升 3.6 个百分点。这进一步说明, 更稳定且更准确的世界模型能够有效提升下游策略学习效果。

表 2: LIBERO 各任务套件上的任务成功率 (%)。

Method	Spatial	Object	Goal	Long	Avg
One-Trajectory SFT					
OpenVLA-OFT [1]	63.6	36.4	48.2	13.8	40.5
w/ GRPO (online) [14]	66.6	45.2	52.2	14.6	44.6
w/ WMPO [20]	67.8	65.4	56.6	13.8	50.9
w/ Ours	84.2	80.8	77.4	35.8	69.5
Δ	+20.6	+44.4	+29.2	+22.0	+29.0
Full-Trajectory SFT					
OpenVLA-OFT [1]	93.6	83.0	90.0	85.6	88.1
w/ GRPO (online) [14]	94.6	86.2	92.2	85.8	89.7
w/ WMPO [20]	95.0	94.8	92.8	87.0	92.4
w/ Ours	98.8	98.8	94.8	91.4	96.0
Δ	+5.2	+15.8	+4.8	+5.8	+7.9

尽管 WoVR 采用了比 OpenSora 更大的 backbone (Wan, 约 5B; OpenSora 约 1.3B), 其推理吞吐仍然更高。这主要是因为 WoVR 只需 5 个 diffusion step, 并利用 3D VAE 进行时空 latent 编码; 相比之下, OpenSora 通常依赖更多采样步数以及 2D VAE。

6.2 4.2 WoVR 能否有效提升 VLA 任务性能?

实验设置。 我们在 LIBERO Spatial、Object、Goal 和 Long 四个 suite 上开展 policy 优化实验 [45], 每个 suite 均包含 10 个任务。遵循 SimpleVLA-RL [17] 的设置, 我们从 OpenVLA-OFT 初始化, 并考虑两种监督微调 (SFT) 设置: one-trajectory SFT 和 full-trajectory SFT。

为保证公平比较, 所有方法在每个 suite 上都使用相同的 2,500 条真实环境轨迹预算。GRPO 直接利用这些轨迹进行在线 policy 优化; WMPO 则用这些数据训练 world model, 随后完全在 imagination 中执行 RL。WoVR 采用第 3.3 节提出的两阶段 policy-simulator 协同演化协议: 首先使用由 base policy 收集的 1,500 条轨迹训练初始 world model, 在 imagination 中优化 policy; 随后, 再利用 evolved policy 额外收集的 1,000 条轨迹对 world model 进行细化。

定量结果。表 2 给出了两种 SFT 初始化下，模型在各个 LIBERO suite 上的成功率（success rate, SR）。在 one-trajectory SFT 设置下，GRPO 仅带来有限提升（4.1%↑），这表明在真实环境交互预算受限时，在线 RL 难以得到有效优化。WMPO 将平均成功率提升至 50.9%，但在 LIBERO-Long 上没有增益（0%↑），说明当 world model 可靠性不足时，它无法支撑长时域的 policy 优化。相比之下，WoVR 在所有 suite 上都取得了最佳表现，将平均成功率提升至 69.5%，其中在 Object 和 Long 上的增益尤为显著。

在更强的 full-trajectory SFT 设置下，base policy 本身已经较强（88.1%），因此各方法的提升空间更小：GRPO 达到 89.7%，WMPO 达到 92.4%，而 WoVR 仍然取得最高的平均成功率 95.9%，并且始终优于 WMPO。

表 3: 真实世界成功率（%，每个任务 30 次试验）。

Method	Franka Emika Panda				AgileX Piper			
	Pick Banana	Pick Bread	Open Drawer	Avg	Fold Towel	Pick Cube	Pick Tomato	Avg
OpenVLA-OFT [1]	36.7	70.0	46.7	51.1	10.0	23.3	13.3	15.5
w/ WMPO [20]	56.7	76.7	60.0	64.5	10.0	26.7	20.0	18.9
w/ Ours	86.7	90.0	63.3	80.0	20.0	33.3	33.3	28.9
Δ	+50.0	+20.0	+16.6	+28.9	+10.0	+10.0	+20.0	+13.4

上述结果表明，基于 world model 的 RL 在有限预算下能够优于在线 RL，尤其当 simulator 足够稳定并且与 policy 保持良好对齐时，这一优势更加明显。关于 PACE 和 KIR 的更多消融实验见附录 D.2。

6.3 4.3 经 WoVR 优化的 policy 能否可靠迁移到真实世界？

实验设置。我们在两种控制特性不同的机器人平台上开展真实世界迁移实验：Franka Emika Panda 和 AgileX Piper。如图 5 所示，我们在每个平台上评估了 3 个 manipulation 任务。各任务的详细要求见附录 F.2。

对于每个平台，我们均围绕这 3 个任务训练一个多任务 VLA policy 及其对应的 world model。在用 WoVR 完成训练后，我们将优化后的 policy 直接部署到真实机器人上，并评估其在真实环境中的成功率（success rate, SR）。每个任务均进行 30 次相互独立的试验。更多细节见附录 F。

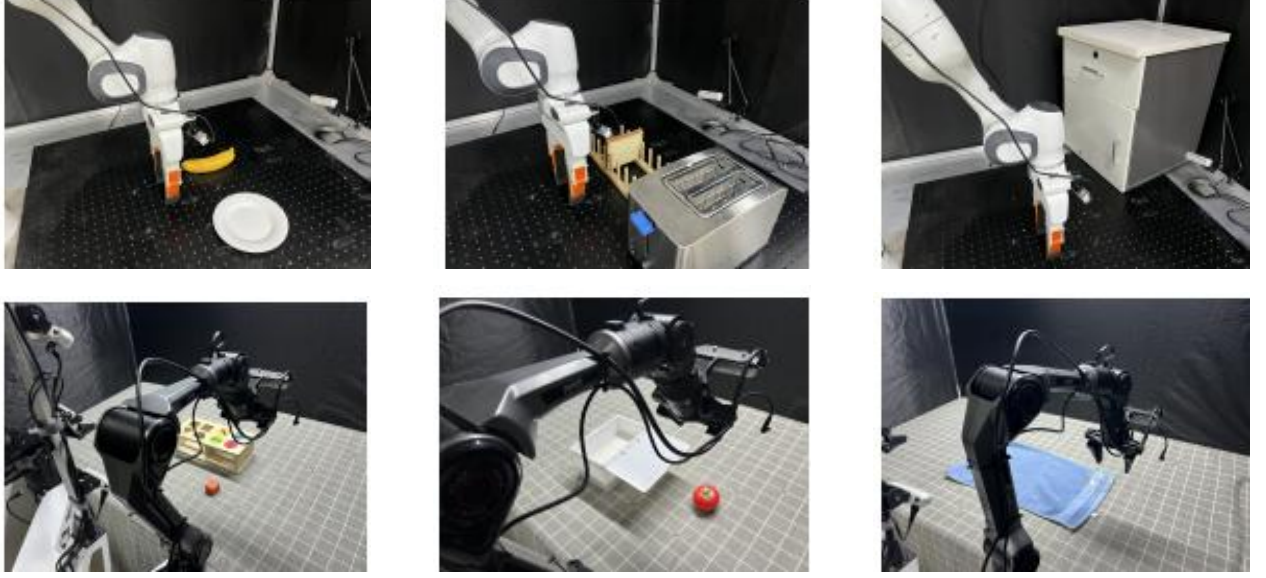


图 5: 两种机器人平台上的真实世界实验。上排: Franka Emika Panda; 下排: AgileX Piper。对应任务分别为: (a) Pick banana, (b) Pick bread, (c) Open drawer, (d) Pick cube, (e) Pick tomato, (f) Fold towel。

定量结果。表 3 汇报了两种机器人平台上的真实世界成功率。WMPO 相比基础 policy 有所提升, 尤其是在 Franka 平台上提升了 13.4 个百分点, 这表明基于 world model 的 policy 优化确实能够提供有价值的学习信号。然而, 在噪声更大的 AgileX Piper 平台上, 其增益仅为 3.4 个百分点, 说明朴素的 imagined RL 仍然容易受到 world-model hallucination 的影响。相比之下, WoVR 始终取得最佳性能: 相对基础 policy, Franka 平台提升 28.9 个百分点, AgileX Piper 平台提升 13.4 个百分点。上述结果表明, WoVR 能够在 imagination 中实现更稳定、更有效的 policy 优化, 并且无需额外的在线交互, 就能强力迁移到高精度和高噪声两类真实机器人系统。

对不同 VLA backbone 的泛化性。为进一步评估 WoVR 的通用性, 我们在 AgileX Piper 平台上额外采用 $\pi_{0.5}$ 作为 VLA backbone 进行实验。我们遵循 π_{RL} [7] 的做法, 在保持 WoVR 框架不变的前提下, 使用 Flow-SDE 对 policy 进行优化。

表 4: 在 AgileX Piper 上, 以 $\pi_{0.5}$ 为 VLA backbone 时的真实世界成功率 (% , 每个任务 30 次试验)。

Method	Fold Towel	Pick Cube	Pick Tomato	Avg
$\pi_{0.5} - \text{base}$	20.0	60.0	23.3	34.4
w/ Ours	30.0	86.7	53.3	56.7
Δ	+10.0	+26.7	+30.0	+22.3

如表 4 所示, WoVR 在所有评测任务上都稳定提升了 $\pi_{0.5}$ policy 的表现, 使平均成功率提高了 22.3 个百分点。后训练得到的 policy 在 pick-and-place 任务上取得了较高成功率。尽管 Fold Towel 是更具挑战性的可变形物体任务, WoVR 仍带来了 +10.0 个百分点的提升。这些结果说明, WoVR 的收益并不依赖于某一种特定的 VLA 架构。

7 5 Conclusion

本文首先指出，在基于 world model 的强化学习（RL）中，用于优化 VLA policy 的闭环想象交互存在 hallucination 问题，这是影响系统可靠性的核心瓶颈。为此，我们提出 WoVR，一个面向 hallucination 感知的框架：通过可由动作控制的视频 world model 提升模拟器稳定性，借助 KIR 降低有效预测深度，并通过 PACE 维持 policy 与模拟器之间的对齐。

在 LIBERO 和真实世界操纵任务上的实验表明，WoVR 能够实现有效的 policy 优化。综合这些结果，我们认为，受控于 hallucination 的 world model 可以作为实用的 RL 模拟器，从而打通一条闭环路径：在线部署数据持续改进模拟器，并进一步提升想象空间中的 VLA policy。

7.1 6 局限性

WoVR 指出，在基于 world model 的强化学习中，closed-loop imagined rollout 里的幻觉（hallucination）是影响可靠性的关键瓶颈，但目前的分析仍以经验研究为主。具体而言，我们尚未形式化刻画幻觉如何传播到 policy 优化过程中，也没有给出一个 regret bound，用于衡量 imagined world model 中学得的 policy 相对于真实环境最优 policy 的性能差距。

此外，尽管 WoVR 在真实世界的桌面操纵任务上展示了较好的效果，但其在显著更长时域任务以及移动操纵场景中的能力仍缺乏充分研究。这类设定可能会因更长的交互时域、持续变化的视角、更大的状态空间以及更为多样的接触动力学，而进一步放大误差累积问题。要将 WoVR 扩展到这类场景，可能需要在更多样的具身数据上开展大规模 world model 预训练，同时设计更有效的机制来抑制误差的逐步累积。

8 参考文献

参考文献条目较多，此处保留原论文英文文献列表，请参阅原文 PDF 的 References 一节。

8.1 附录目录

A GPU 分配策略	15
B 实现细节	15
B.1 world model 的实现细节	15
B.2 Reward Model 的实现细节	16
B.2.1 稀疏奖励模型	16
B.2.2 稠密奖励建模	16
B.3 KIR 的实现细节	17
C 评估指标	17
D 消融研究	18
D.1 world model 机制的消融	18
D.2 PACE 与 KIR 的消融	19
D.3 奖励建模的消融	21
E 定性失败模式分析	22
F 真实世界实验	23

F.1 硬件配置	23
F.2 任务需求细节	23
F.3 真实世界部署中的实际注意事项	23
F.4 任务设置	25

8.2 一种 GPU 分配策略

强化学习流水线可以分解为三个组成部分：Generation、Simulator 和 Training。在 WoVR 中，Simulator 由学习得到的 world model 实现：它根据当前观测和动作生成下一时刻观测。在 rollout 阶段，Generation 基于当前观测执行 policy inference，输出动作（可选地采用 action chunking）；随后，Simulator 执行动作并返回下一时刻观测。这样的 closed-loop 交互会不断重复，直到收集到一批轨迹为止。在优化阶段，Training 利用收集到的轨迹更新 VLA policy；完成后，系统再切回 rollout，进入下一轮迭代。

遵循 RLinf-VLA 的系统抽象，WoVR 采用一种 collocated（共享式）GPU 分配策略，即强化学习流水线中的三个组件共同驻留在同一组 GPU 上，其中 Simulator 具体由 world-model rollout 模块实现。与需要专用设备侧状态的物理模拟器不同，WoVR 的 simulator 本质上是一个神经网络。因此，它的 offload/onload 可以自然地通过仅交换模型参数在 GPU 与主机内存之间实现，而无需迁移任何外部 simulator 状态。

在其原始形式下，collocated 执行依赖频繁的 GPU↔CPU offload/onload，从而保证任一时刻只有一个组件驻留在 GPU 上。然而，在具身场景中，simulator 与 generator 必须进行迭代式交互，如果每次交互都执行一次 offload/onload，代价将高得难以接受。因此，我们采用了修改后的 collocated 策略：Generation 与 Simulator 的 offload/onload 仅在 rollout 阶段的开始和结束时发生，从而避免 closed-loop imagined interaction 过程中的重复传输。其示意如图 6 所示。

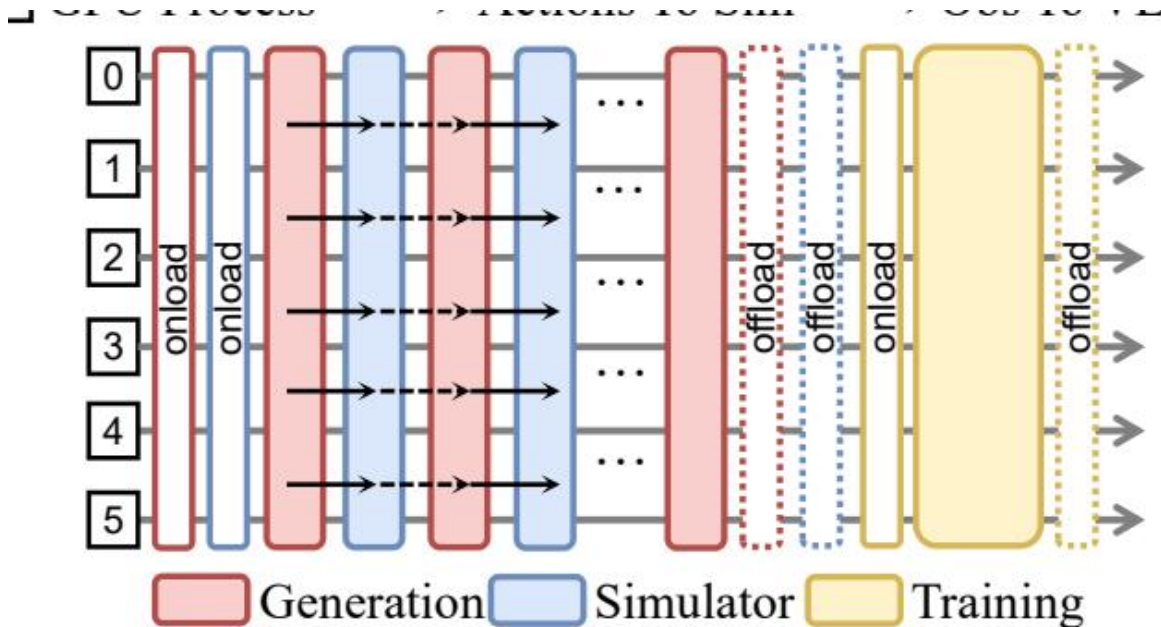


图 6: 共享式（collocated）GPU 分配策略示意图

9 B 实现细节

9.1 B.1 world model 的实现细节

world model 在视频 diffusion backbone 的潜空间中，以 chunk 为单位逐段生成 imagined trajectory。在 rollout 的第 t 步，锚定视觉上下文由该 episode 的初始帧与最新生成的记忆帧拼接而成，即 $[o_0, o_{t-c:t}]$ 。经过 VAE 编码后，得到对应的潜变量上下文 $[z_0, z_{t'-c':t'}]$ ，再与下一个 chunk 的高斯噪声潜变量拼接。随后，带动作条件的 DiT blocks 在图 3 所示的双通道动作条件下对这些潜变量执行去噪，并将解码后的未来帧追加到 rollout 上下文中。重复这一过程，即可在 policy 动作驱动下生成成长时域、closed-loop 的 imagined trajectory。

我们采用 Rectified Flow 目标函数 [46] 训练该模型。令 $x_1 = z_{t'+1:t'+H'}$ 表示目标未来潜变量，令 $x_0 \sim \mathcal{N}(0, \mathbf{I})$ 表示高斯噪声。给定插值时间 $\tau \in [0, 1]$ ，线性插值状态定义为：

$$x_\tau = (1 - \tau)x_0 + \tau x_1. \quad (2)$$

沿该插值轨迹对应的目标速度场为：

$$v_\tau = \frac{dx_\tau}{d\tau} = x_1 - x_0. \quad (3)$$

模型 $u(\cdot; \phi)$ 根据中间潜变量 x_τ 预测速度 v_τ ：

$$[\text{formula}] \quad (4)$$

其中， c 包含锚定视觉上下文和动作序列。

为提升 closed-loop inference 时的鲁棒性，我们在训练阶段向非参考上下文潜变量中注入 diffusion noise：

$$z_{t'-c':t'} = \tau_{ctx} * z_{t'-c':t'} + (1 - \tau_{ctx}) * \epsilon, \quad (5)$$

其中， τ_{ctx} 是一个接近 1 的数值， ϵ 表示噪声。初始参考帧保持干净，而最近的记忆潜变量在作为上下文使用前会被轻微扰动。该数据增强能够缩小由“以模型自生成帧作为条件”所带来的训练-推理分布差异，并缓解长时域自回归 rollout 中脆弱的上下文复制问题 [47, 31]。

9.2 B.2 Reward Model 的实现细节

9.3 B.2.1 稀疏奖励模型

对于稀疏监督，我们采用一个学习得到的奖励分类器，根据预测的下一时刻观测来判断任务是否成功。具体而言，给定 world model 生成的观测 $\tilde{o}_{t+1}$ ，稀疏奖励模型 R_ψ^{sp} 用于估计任务成功的概率，并将二值奖励定义为

$$r_{t+1}^{\text{sp}} = \mathbb{I}\left(R_\psi^{\text{sp}}(\tilde{o}_{t+1}) \geq 0.5\right), \quad (6)$$

其中， $\mathbb{I}(\cdot)$ 表示指示函数。按照 HiL-SERL [16] 的做法，稀疏奖励分类器由一个轻量级网络实现，并在带有成功状态标注的数据上使用二元交叉熵损失进行训练，其定义为

$$\mathcal{L}_{\text{binary}}(\psi) = -\frac{1}{N} \sum_{i=1}^N \left[r_i \log C_\psi(x_i) + (1 - r_i) \log (1 - C_\psi(x_i)) \right]. \quad (7)$$

9.4 B.2.2 稠密奖励建模

为了在二值成功信号之外提供更细粒度的学习信号，我们进一步在 Qwen3-VL [48] 之上引入了一个稠密奖励模型。与直接回归一个标量奖励不同，我们将稠密奖励预测表述为一个有序的视觉进度估计问题。给定四个连续观测 $\tilde{o}_{t-2:t+1} = [\tilde{o}_{t-2}, \tilde{o}_{t-1}, \tilde{o}_t, \tilde{o}_{t+1}]$ 以及任务描述 l ，稠密奖励模型 R_ψ^{de} 预测一个离散的进度等级

$$\ell_t \in \{0, 1, \dots, 10\}, \quad (8)$$

其中，0 表示没有进展，10 表示任务完成。随后通过归一化得到稠密奖励：

$$r_t^{\text{de}} = \frac{\ell_t}{10}. \quad (9)$$

该模型以四帧连续 RGB 图像和任务文本作为输入，并输出 11 个奖励等级上的 logits。在结构设计上，我们采用带有 LoRA 适配的 Qwen3-VL backbone，并额外接入一个轻量级 MLP 奖励头来完成分类。训练过程中，标签由轨迹奖励自动构造：失败轨迹被赋为等级 0，成功状态被赋为等级 10，而成功之前的状态则根据其第一个成功状态在时间上的接近程度，线性映射到中间等级 1-9。为缓解类别不平衡问题，我们在数据集构建阶段进一步对不同奖励等级的样本进行了重平衡。

稠密奖励模型采用标准的交叉熵目标进行训练：

$$\mathcal{L}_{\text{dense}}(\psi) = \mathbb{E}_{(\tilde{o}_{t-2:t+1}, d, \ell_t)} \left[-\log p_\psi^{\text{de}}(\ell_t | \tilde{o}_{t-2:t+1}, d) \right]. \quad (10)$$

9.5 B.3 KIR 的实现细节

Keyframe-Initialized Rollouts (KIR) 的实现方式，是修改输入给 world model 的初始上下文，而不是修改模型架构。我们在 world model 中使用的 Wan encoder 是一个因果 3D VAE，其时间维输入长度满足 $1 + 4m$ 的形式。在我们的 rollout 实现中，总共使用 5 帧 ($m = 1$)：1 个持续保留的参考帧，以及 4 个时间上下文帧。

对于从 episode 起始时刻开始的标准长时域 rollout，我们通过重复第一帧来构造合法的 VAE 输入：

$$\mathcal{C}^{\text{init}} = [o_0, o_0, o_0, o_0, o_0] = \text{Repeat}(o_0, 5), \quad (11)$$

对应生成的第一个 chunk 为

$$\tilde{o}_{1:9} = \mathcal{W}_\phi(\mathcal{C}^{\text{init}}, a_{0:8}), \quad (12)$$

其中， $\mathcal{W}_\phi$ 表示 action-conditioned world model。等价地，在与任务特定时域上的 KIR rollout 做比较时，非 KIR 的上下文可以写为

$$\tilde{O}_{1:9} = \mathcal{W}_\phi(\text{Repeat}(o_0, 5), a_{0:8}), \quad (13)$$

这种写法去除了失败发生前的局部上下文，仅依赖初始观测。这样的初始化会迫使 world model 在到达任务关键状态之前，先“想象”完整的前缀过程，从而增大误差累积的有效深度。

与此不同，KIR 会在接近任务关键状态的位置初始化 imagined rollout。理想情况下，对于一条将在 $T_{\text{KIR}} + 1$ 附近发生失败的轨迹，我们会使用第一帧以及失败转移前的最后 4 帧：

$$\mathcal{C}_T^{\text{KIR}} = [o_0, o_{T_{\text{KIR}}-3}, o_{T_{\text{KIR}}-2}, o_{T_{\text{KIR}}-1}, o_{T_{\text{KIR}}}], \quad (14)$$

并基于这一 keyframe 上下文生成第一个 imagined chunk:

$$\tilde{o}_{T_{\text{KIR}}+1:T_{\text{KIR}}+9} = W_\phi(\mathcal{C}_T^{\text{KIR}}, a_{T_{\text{KIR}}:T_{\text{KIR}}+8}). \quad (15)$$

其中, 第一帧 o_0 仍作为全局锚点, 而 $o_{T_{\text{KIR}}-3:T_{\text{KIR}}}$ 则提供失败前的局部上下文。

在实际实现中, 我们不会为每条轨迹手动挑选 keyframe。对于每个任务, 我们设置一个任务级的 keyframe 索引 T_{KIR} , 其依据是 base policy 通常何时接近关键失败区域。这样的固定任务级选择使 KIR 成本较低、易于复现, 同时仍能让第一个 imagined chunk 落在任务的决定性阶段附近。在生成第一个 KIR chunk 之后, 后续 chunk 与标准 world-model rollout 采用相同的自回归过程: 使用第一帧锚点以及最近生成的 memory 帧作为上下文。

9.6 C 评估指标

我们采用 LPIPS [39]、FID [40]、FVD [41]、FloLPIPS [42] 和 FPS 作为生成视频的评估指标。具体如下:

- LPIPS (Learned Perceptual Image Patch Similarity) 基于深层视觉特征度量帧级感知相似性:

$$d(x, x_0) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \left\| w_l \odot \left(\hat{F}_{h,w}^l(x) - \hat{F}_{h,w}^l(x_0) \right) \right\|_2^2 \quad (16)$$

其中, x 和 x_0 分别表示生成帧与真实帧; $\hat{F}_{h,w}^l(\cdot)$ 表示第 l 层在空间位置 (h, w) 处的归一化深层特征, w_l 为学习得到的通道级权重。

- FID (Frechet Inception Distance) 基于图像级特征统计量, 评估生成帧与真实帧之间的分布相似性:

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{Tr} \left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2} \right) \quad (17)$$

其中, (μ_r, Σ_r) 和 (μ_g, Σ_g) 分别表示从真实帧和生成帧中提取的图像级特征的均值与协方差。

- FVD (Frechet Video Distance) 将 FID 的形式从图像级特征扩展到视频级特征。它通过对时空视频表征的均值和协方差进行统计, 评估真实视频与生成视频之间的分布相似性:

$$\text{FVD} = \mu_r^v - \mu_g^v + \text{Tr} \left(\Sigma_r^v + \Sigma_g^v - 2(\Sigma_r^v \Sigma_g^v)^{1/2} \right) \quad (18)$$

其中, (μ_r^v, Σ_r^v) 和 (μ_g^v, Σ_g^v) 分别表示从真实视频和生成视频中提取的视频级特征的均值与协方差。

- FloLPIPS 在估计得到的 optical flow 轨迹上进行运动对齐后, 度量感知相似性:

$$\text{FloLPIPS} = \frac{1}{T-1} \sum_{t=1}^{T-1} \sum_l \frac{\sum_{h,w} M_t(h, w) \left\| w_l \odot \left(\hat{F}_{h,w}^l(V_t) - \hat{F}_{h,w}^l(\hat{V}_t) \right) \right\|_2^2}{\sum_{h,w} M_t(h, w)} \quad (19)$$

其中, 运动权重定义为:

$$M_t(h, w) = \left\| \text{Flow}(V_t, V_{t+1})_{h,w} - \text{Flow}(\hat{V}_t, \hat{V}_{t+1})_{h,w} \right\|_2 \quad (20)$$

这里, $\hat{V}_t$ 和 V_t 分别表示时刻 t 的生成帧与真实帧。 $\hat{F}_{h,w}^l(\cdot)$ 表示感知网络在第 l 层、空间位置 (h, w) 处提取的归一化特征, w_l 是该层学习得到的感知权重。 $\text{Flow}(V_t, V_{t+1})$ 和 $\text{Flow}(\hat{V}_t, \hat{V}_{t+1})$ 分别表示真实视频与生成视频中相邻两帧之间的 optical flow。 $M_t(h, w)$ 基于 optical flow 度量位置 (h, w) 处真实视频与生成视频在运动模式上的差异。

- FPS 表示每秒可生成的帧数, 用于量化 world model 的生成效率。

10 D Ablation Study

10.1 D.1 世界模型机制的消融实验

我们首先对所提出世界模型中的核心设计选择进行消融研究, 目的是理解不同的上下文建模机制如何影响长时域视频生成的稳定性。具体而言, 我们考察以下因素: (i) 作为视觉上下文的 memory frame 数量; (ii) 是否使用固定参考帧 (fixed reference frame); (iii) 训练过程中对上下文帧加入噪声的影响。

实验变体 我们将完整的 WoVR 模型与以下三种消融变体进行比较:

- **WoVR w/o ref**: 从上下文窗口中移除固定参考帧;
- **WoVR w. mem=1**: 仅使用单帧上下文;
- **WoVR w/o noisy context**: 在训练时关闭对上下文帧的噪声注入。

所有变体均仅在 LIBERO-Spatial 基准上训练与评估。我们使用 1,500 条 VLA rollout 轨迹训练该世界模型, 并在一个包含 24 条轨迹的留出测试集上进行评估。

定量结果 表 5 汇报了在不同 rollout 时域长度下, 以 LPIPS、FID、FVD 和 FloLPIPS 衡量的定量结果。与仅使用单帧上下文相比, 采用带固定参考锚点的多帧上下文在所有指标上都带来了显著提升。

为进一步理解这些定量趋势背后的失效模式, 我们在图 7 中给出了定性对比。图中可以看到, 不使用固定参考帧或不使用 noisy context 的模型, 在长时域 rollout 中会出现明显的空间漂移以及目标消失; 相比之下, 完整的 WoVR 模型在视觉上保持了更好的稳定性, 并且与真实结果更加一致。

表 5: 世界模型机制的消融实验 (LIBERO-Spatial)。Rollout 表示 rollout 时域长度。

Method	Rollout	Metrics			
		LPIPS↓	FID↓	FVD↓	FloLPIPS↓
WoVR (Ours)	512	0.091	36.687	73.493	0.154
	256	0.069	27.238	63.948	0.110
	128	0.051	20.780	49.017	0.081
WoVR w/o ref	512	0.133	73.942	123.502	0.168
	256	0.089	49.406	86.000	0.116
	128	0.064	35.559	86.146	0.090
WoVR w. mem=1	512	0.120	64.501	86.042	0.165
	256	0.086	46.790	81.742	0.117
	128	0.065	36.047	79.605	0.095
WoVR w/o noisy context	512	0.099	44.712	77.284	0.160
	256	0.074	31.691	61.660	0.115
	128	0.054	23.444	58.836	0.085

移除参考帧会导致性能明显下降，尤其是在 rollout 时域更长时更为突出。这一结果表明，利用固定参考帧对上下文进行锚定，能够有效抑制自回归反馈环中的误差累积，而这对于维持长时域视频生成的稳定性至关重要。

此外，关闭对上下文帧的噪声注入同样会带来可观的性能下降。尽管在短 rollout 下这种退化相对温和，但随着 rollout 长度增加，二者之间的差距会愈发明显。这说明，对上下文帧加入轻微噪声能够通过降低模型对精确条件输入的过度依赖，提升长时域生成过程中的鲁棒性，从而缓解训练与推理之间的 gap。

总体来看，这些结果表明，本文提出的上下文建模策略——结合固定参考帧、多帧 memory window 以及 noisy context 增强——对于稳定长时域视频生成起到了关键作用。这些机制共同使 WoVR 能够在 closed-loop 自回归推理过程中保持较高保真度与时序一致性，从而为下游强化学习提供一个更可靠的模拟器。

10.2 D.2 在 PACE 与 KIR 上的消融实验

我们进一步对 WoVR 中两个交互层面的可靠性机制进行消融：PACE 用于将 world model 与更新后的 policy 分布对齐；KIR 则将 imagined rollout 初始化在靠近任务关键状态的位置，以减少有效预测深度。该消融实验旨在区分两类收益：一类来自收集与 policy 对齐的数据，另一类来自以关

键帧初始化的 imagined interaction。

实验设置. 我们在第 4.2 节所述的 one-trajectory SFT policy optimization 协议下, 比较四种 WoVR 变体:

- **WoVR w/o PACE (1,500 base):** world model 仅使用由 base policy rollout 得到的 1,500 条轨迹进行训练;
- **WoVR w/o PACE (2,500 base):** world model 使用由 base policy rollout 得到的 2,500 条轨迹进行训练, 在不改变 policy 分布的前提下增加数据量;
- **WoVR w/o KIR:** 保留 PACE 的数据协议, 但移除关键帧初始化, 因此 imagined rollout 从 episode 起始处开始初始化。

结果. 表 6 给出了在 LIBERO-Spatial 和 LIBERO-Object 上完成的消融结果。WoVR 在两个测试套件上均取得最佳表现, 在这两个已报告套件上的平均成功率 (success rate) 达到 82.5%。去除 PACE 的两个变体用于隔离额外 base-policy 数据的作用: 当 world-model 训练集从 1,500 条 base-policy 轨迹增加到 2,500 条时, 两个套件的平均成绩仅从 75.8% 提升到 77.5%。这一有限的 +1.7 个百分点增益表明: 当同一 base-policy 分布已被较充分覆盖后, 仅仅从该分布中继续增加数据, 所带来的收益是有限的。

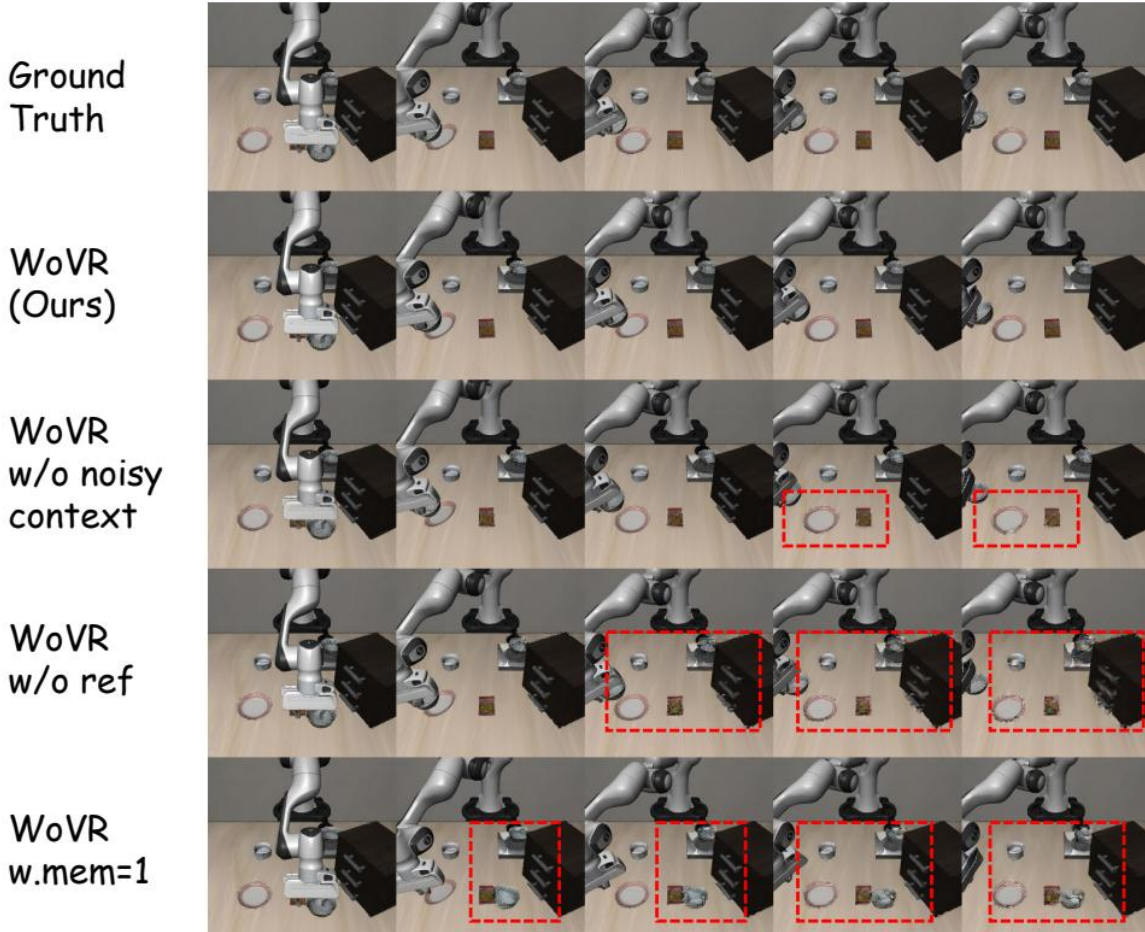


图 7: LIBERO-Spatial 上的定性消融结果。消融变体在长时域 rollout 下会出现误差累积与视觉漂移, 而完整的 WoVR 模型则保持稳定, 并与真实轨迹一致。

表 6: PACE 与 KIR 的消融实验。所有变体均通过 imagined rollout 优化 policy。Avg 为两个已报告套件上的平均值。

Method	Spatial	Object	Avg $\uparrow$
One-Trajectory SFT			
OpenVLA-OFT-base [1]	63.6	36.4	50.0
WoVR w/o PACE (1,500 base)	75.4	76.2	75.8
WoVR w/o PACE (2,500 base)	77.8	77.2	77.5
WoVR w/o KIR	81.6	77.8	79.7
WoVR (1,500 base + 1,000 aligned)	84.2	80.8	82.5
Δ	+20.6	+44.4	+32.5

图 8 展示了 WoVR 在 one-trajectory SFT 训练过程中的演化。PACE 切换之前, imagined training success rate 可以持续提升, 而真实环境中的评估曲线却明显滞后, 这揭示出不断演化的 policy 与 base world model 之间的失配正在加剧。将模拟器从 WM_{Base} 更新为 WM_{Evo} 之后, imagined training 曲线被重新校准到由 policy 诱导的分布上, 并与真实环境评估结果更加一致。这种 PACE 之后的对齐表明: 基于演化后 policy 收集的 rollout 数据, 有助于纠正模拟器中累积的失配, 并缓解 imagined RL 中由 hallucination 引发的过度乐观现象。结合表 6 中的定量结果, 这些曲线共同支持了 PACE 的作用, 即使 world-model 训练信号更忠实地反映真实世界中的 policy 性能。



图 8: WoVR 下 one-trajectory SFT 的训练与真实评估曲线。蓝色表示 world model 中的训练成功率，红色表示真实环境中的评估成功率，竖向阴影带标示了从 WM_{Base} 到 WM_{Evo} 的 PACE 切换。



图 9: WoVR 下 one-trajectory SFT 的训练与真实评估曲线。蓝色表示 world model 中的训练成功率，红色表示真实环境中的评估成功率，竖向阴影带标示了从 WM_{Base} 到 WM_{Evo} 的 PACE 切换。



图 10: WoVR 下 one-trajectory SFT 的训练与真实评估曲线。蓝色表示 world model 中的训练成功率，红色表示真实环境中的评估成功率，竖向阴影带标示了从 WM_{Base} 到 WM_{Evo} 的 PACE 切换。

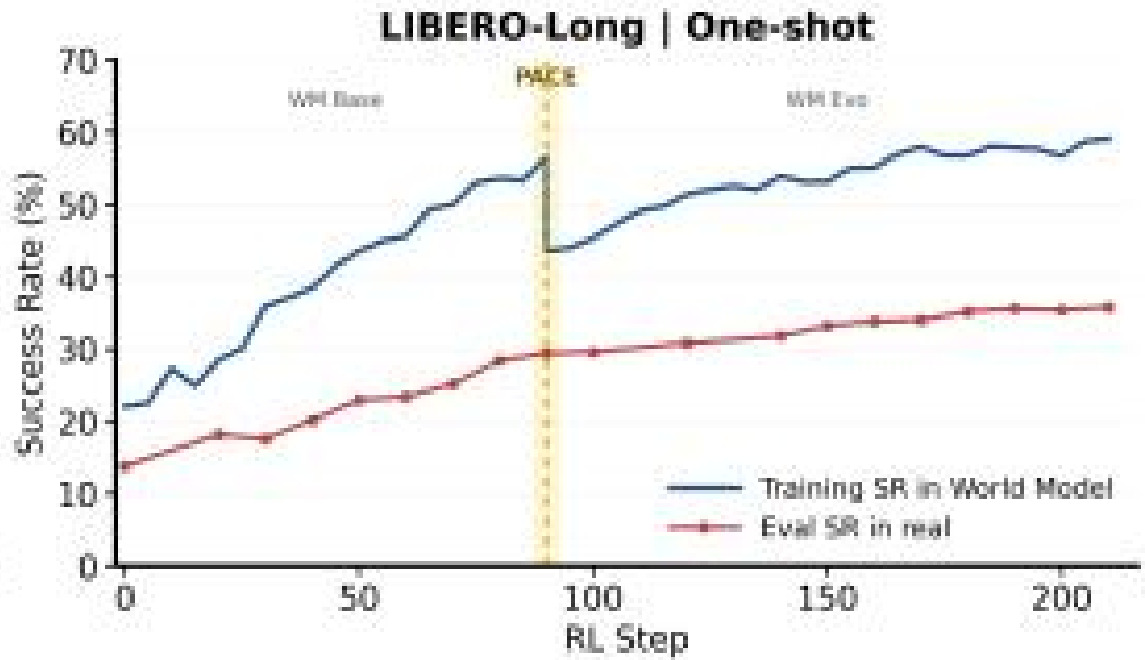


图 11: WoVR 下 one-trajectory SFT 的训练与真实评估曲线。蓝色表示 world model 中的训练成功率，红色表示真实环境中的评估成功率，竖向阴影带标示了从 WM_{Base} 到 WM_{Evo} 的 PACE 切换。

10.3 D.2 PACE 与 KIR 的消融实验（第二部分）

KIR 的消融结果进一步表明：将 imagined rollout 初始化在接近任务关键状态的位置是十分重要的。移除 KIR 后，Spatial 上的性能从 84.2% 降至 81.6%，Object 上从 80.8% 降至 77.8%，两个测

试套件的平均值也从 82.5% 降至 79.7%。这些结果说明，即便保留 PACE，KIR 仍能提升 imagined interaction 的效果；其原因在于，KIR 缩短了有效预测深度，并减少了早期误差累积。综合 PACE 与 KIR 的消融结果可以看出，WoVR 同时受益于与 policy 对齐的 simulator 优化，以及由关键帧初始化的 rollout 设计。

10.4 D.3 奖励建模的消融实验

我们进一步分析不同奖励建模选择对 policy 优化的影响，并比较了两种方案：一种是基于轻量级 ResNet 的稀疏奖励模型，另一种是基于 Qwen3-VL 的稠密奖励模型。

实验设置. 我们固定其余所有组件，仅改变 policy 优化过程中使用的奖励模型。稀疏奖励模型预测二值任务是否成功，而稠密奖励模型提供更细粒度的任务进展信号。两种变体均采用与第 4.2 节相同的训练设置与评估指标。

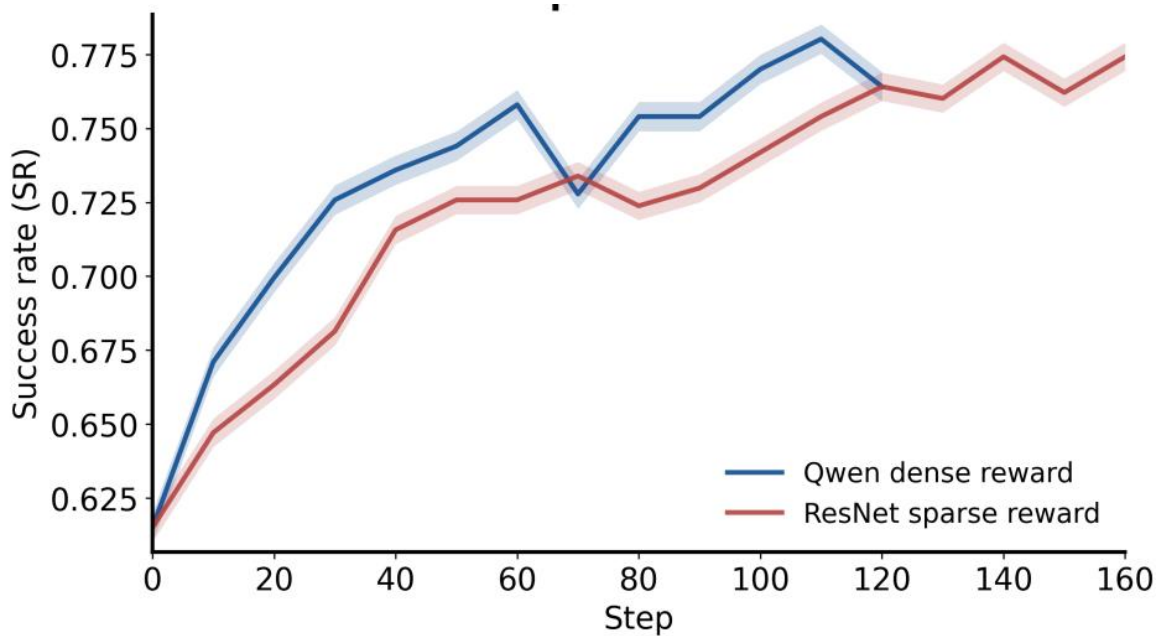


图 12: 奖励建模的消融实验：稠密奖励与稀疏奖励的对比。

消融：稠密奖励 vs 稀疏奖励.

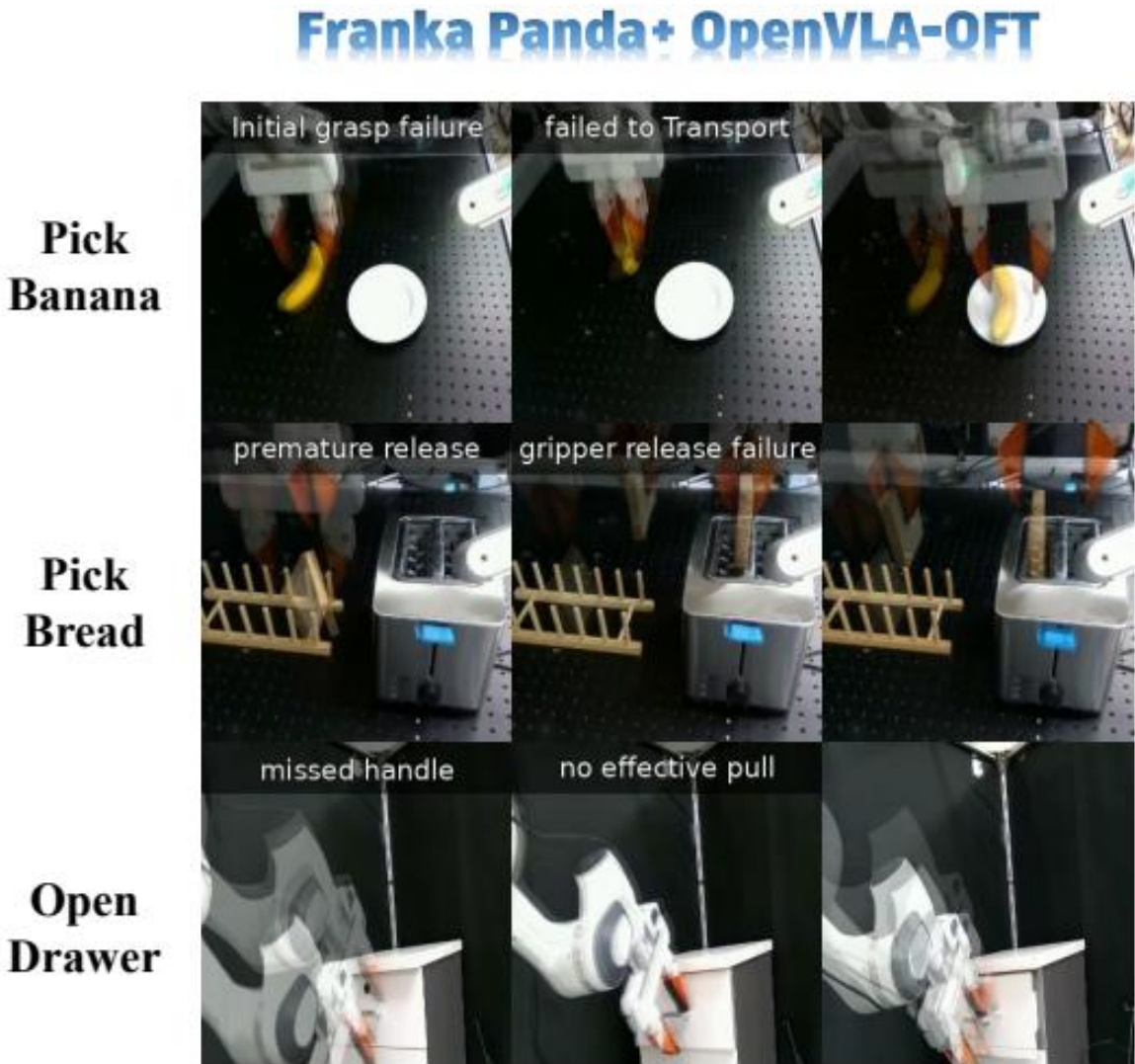
结果与分析. 图 9 展示了两种奖励设定下的学习曲线。可以看到，稠密奖励模型在训练早期提升了样本效率，从而带来更快的初始性能增长。然而，两种奖励设计最终收敛到的性能水平相近。

尽管稠密奖励模型在早期学习阶段具有优势，但其计算开销显著更高，因为它依赖于一个大规模 vision-language backbone (Qwen3-VL 2B)。在实际实验中我们发现，使用稠密奖励模型进行 rollout 的速度，相比轻量级稀疏奖励模型约慢 3 $\times$ 。

AgileX Piper + OpenVLA-OFT

此外，二值奖励已被广泛用于 LIBERO 等 manipulation benchmark，以及真实机器人系统中。综合以上因素，在所有主要实验中我们均采用稀疏奖励模型。

11 E 定性失效模式分析



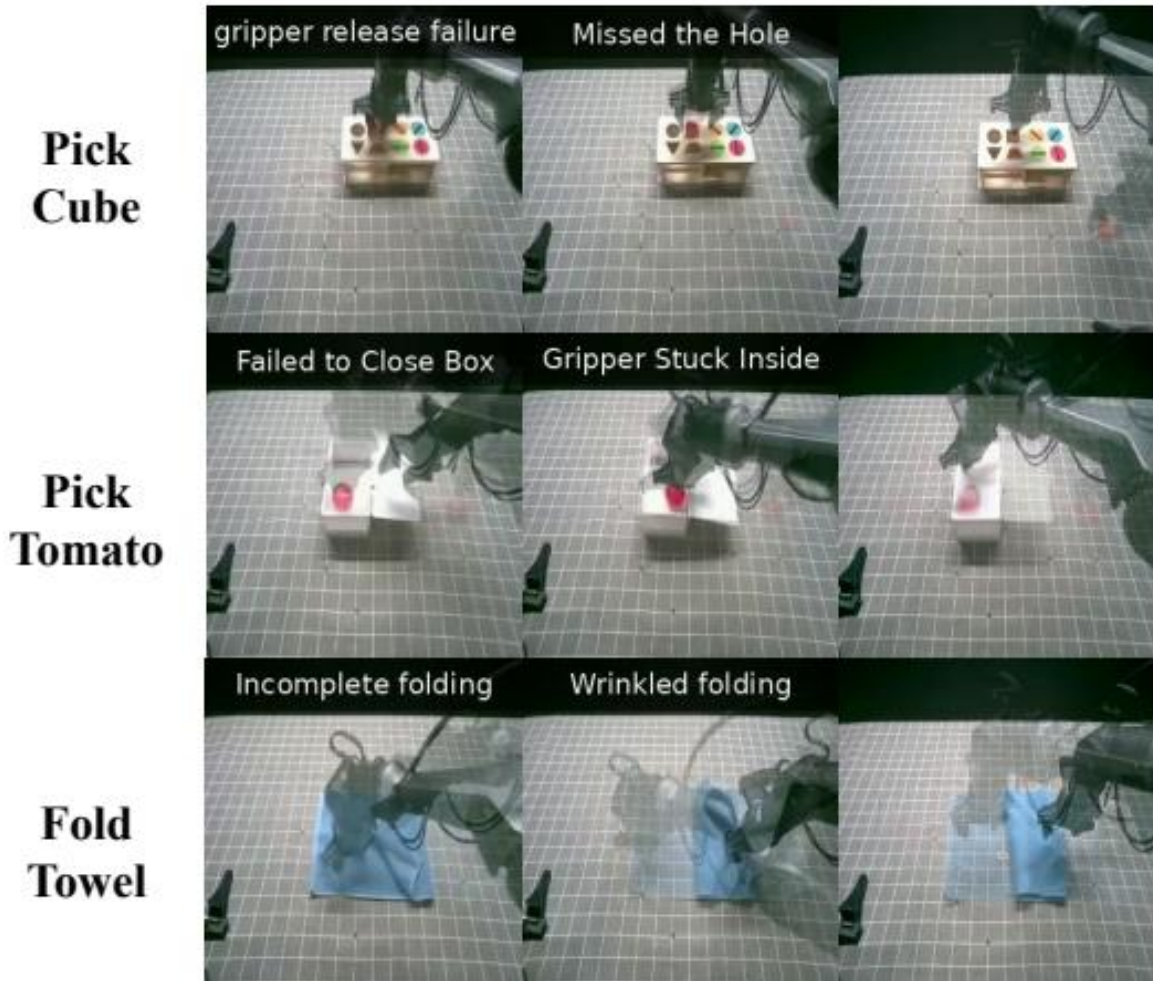


图 13: WoVR policy 优化前后真实世界执行过程的可视化。

我们进一步分析了基础 VLA 模型的代表性失效模式，并将其与 WoVR 优化后的行为进行比较。图 10 展示了两个机器人平台、六项操纵任务上的执行轨迹。对比结果表明，WoVR 不仅提升了任务成功率 (success rate)，还改变了基础 VLA policy 的主要失效模式。

在 Franka Panda 平台上，基础 policy 在三项任务中表现出不同的失效模式。在 Pick Banana 任务中，机器人常常反复尝试抓取香蕉，却始终无法形成稳定抓握；另一类情况是虽然成功抓起了香蕉，但未能将其移动到盘子上方。应用 WoVR 后，policy 能更稳定地抓取香蕉，并以更快、更直接的运动将其转移到盘中。在 Pick Bread 任务中，基础 policy 的主要失败原因包括：gripper 过早张开，导致面包掉落；或者到达目标区域后仍保持 gripper 闭合。WoVR 在很大程度上抑制了这两类失效模式，使释放行为更加稳定一致。在 Open Drawer 任务中，基础 policy 经常无法准确到达把手位置，或者在接触把手后未能继续拉开抽屉。使用 WoVR 后，机器人更常能够完成完整的交互序列，包括接近把手、建立接触以及拉动抽屉。

在 AgileX Piper 平台上，失效更容易受到状态估计噪声较大以及底层控制稳定性较差的影响。在 Pick Cube 任务中，基础 policy 往往能将立方体移动到目标区域，但无法释放 gripper；或者虽完成放置，却因姿态不正确而未能落入目标孔中。WoVR 通过生成更稳定的放置与释放行为，显著减少了这些错误。在 Pick Tomato 任务中，基础 policy 常见的失败情况包括：放入番茄后未能关闭盒盖，或者 gripper 被卡在盒内。经过 WoVR 优化后，这些失效现象有所缓解，但该任务仍然对接触过程和盒体几何较为敏感。在 Fold Towel 任务中，基础 policy 经常无法完成折叠，或者在折叠后

仍持续抓住毛巾。WoVR 改善了折叠与释放之间的时序协调，因此在这一可变形物体任务上取得了更高的成功率。

总体来看，这一定性分析表明，WoVR 通过减少反复出现的动作层面失效模式，提升了真实世界执行表现。这些失效模式包括抓握不稳定、gripper 释放过早或过晚、目标放置不准确，以及接触丰富交互过程不完整等。上述结果与第 4.3 节报告的真实世界定量改进相一致，也说明：在 imagination 中进行具备 hallucination 感知能力的 policy 优化，能够迁移为物理机器人上更可靠的 closed-loop 行为。



图 14: (a) AgileX Piper



图 15: (b) Franka Emika Panda

图 16: 两类机器人机械臂平台。

11.1 F 真实世界实验

11.2 F.1 硬件配置

如图 11 所示, 我们在两种不同的真实机器人平台上评估所提出的方法: 左侧为 AgileX Piper 机械臂, 右侧为 Franka Emika Panda 机械臂。两套平台均配备双指 gripper。采用这两种平台的目的是在不同控制精度的硬件条件下检验方法的鲁棒性。

AgileX Piper AgileX Piper 是一款 6-DoF 机械臂。我们基于 cobot Magic 平台进行远程操作 (tele-operation)。视觉输入采用头部视角的 D435 相机采集 RGB 图像，并以 10Hz 的频率对数据进行同步与记录。

Franka Emika Panda Franka Emika Panda 是一款 7-DoF 机械臂。我们使用固定的第三人称视角 D435 相机记录 RGB 图像，并使用 spatial mouse 进行远程操作。数据以 30Hz 的频率记录。

11.3 F.2 任务的详细要求

下面给出两个机器人平台上各项任务的详细要求。

在 Franka 平台上，我们评测三项操纵任务：

- **Pick Banana**: 抓取一根香蕉，并将其放到盘子上；
- **Pick Bread**: 抓取一片面包，并将其放入烤面包机中；
- **Open Drawer**: 将抽屉拉开；

在 AgileX Piper 平台上，我们还评测另外三项任务：

- **Pick Cube**: 抓取一个立方体，并将其放入盒中；
- **Pick Tomato**: 抓取一个番茄，将其放入盒中，并盖上盒盖；
- **Fold Towel**: 将毛巾折叠成紧凑形态。

11.4 F.3 面向真实世界部署的实践考量

这里介绍机械臂的 action space 与控制方式选择。这一设计对于实际运行中精度较低的机械臂尤为关键。

平台与 action space. 实验在两个机器人平台上开展：AgileX Piper manipulator 和 Franka Emika Panda。对于两个平台，action space 都定义在 delta end-effector (delta EE) 空间中，即 VLA policy 输出末端执行器的相对位移；这些相对动作同样作为 world model 的输入。

由于两种机械臂的底层驱动都在绝对关节空间中控制，因此需要先将每个预测的 delta action 转换为绝对末端位姿，再通过逆运动学 (IK) 求得可执行的关节命令。

旋转参数化. 旋转动作的参数化方式是一个关键设计选择。直接回归 ΔRPY 往往不稳定，因为欧拉角存在不连续性与奇异性，而且很小的预测误差也可能在重建绝对朝向时被放大，从而影响 IK 的一致性。

为提升鲁棒性，本文采用 Rotation 6D 表示 [49, 50]。给定当前末端执行器朝向 $R_t \in SO(3)$ 和下一时刻朝向 $R_{t+1} \in SO(3)$ ，首先计算局部相对旋转：

$$\Delta R_t = R_t^\top R_{t+1}. \quad (21)$$

按照标准的 6D 旋转表示，RotMatTo6D($\cdot$) 表示将旋转矩阵的前两列展平。对应的 6D 形式 delta rotation 计算如下：

$$\Delta \mathbf{r}_{6D,t} = \text{RotMatTo6D}(\Delta R_t) = \left[\Delta R_t[:, 1]^\top, \Delta R_t[:, 2]^\top \right]^\top \in \mathbb{R}^6. \quad (22)$$

其中, $\Delta \mathbf{r}_{6D,t}$ 表示在局部 end-effector 坐标系下, 从当前位姿到下一位姿的相对旋转。

执行时, 需要将预测得到的 6D 向量恢复为合法的旋转矩阵, 这里采用 Gram-Schmidt 正交化。具体地, 给定两个预测向量 $\mathbf{a}_1, \mathbf{a}_2 \in \mathbb{R}^3$, 计算:

$$\mathbf{b}_1 = \frac{\mathbf{a}_1}{\|\mathbf{a}_1\|}, \quad (23)$$

$$\mathbf{b}_2 = \frac{\mathbf{a}_2 - (\mathbf{b}_1^\top \mathbf{a}_2) \mathbf{b}_1}{\|\mathbf{a}_2 - (\mathbf{b}_1^\top \mathbf{a}_2) \mathbf{b}_1\|}, \quad (24)$$

$$\mathbf{b}_3 = \mathbf{b}_1 \times \mathbf{b}_2, \quad \widehat{\Delta R}_t = [\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3]. \quad (25)$$

随后, 将绝对朝向更新为:

$$\widehat{R}_{t+1} = R_t \widehat{\Delta R}_t. \quad (26)$$

与 RPY 相比, Rotation 6D 避免了角度回绕 (angle wrapping) 和 gimbal-lock 奇异性; 与四元数相比, 它避免了双覆盖歧义, 即 q 与 $-q$ 表示同一旋转。此外, 网络可以直接回归一个无约束的 6D 向量, 之后再通过正交化投影为合法旋转矩阵。这使得动作表示更加连续, 也更适合学习稳定的 end-effector 旋转命令。

最终的 action space 定义为:

$$(\Delta x, \Delta y, \Delta z, \Delta \mathbf{r}_{6D}, \text{gripper}).$$

平台相关考量. 尽管整体控制流水线在两个平台上是共享的, 但它们的估计精度并不相同。

Franka Panda 能够提供高精度且稳定的 end-effector 状态反馈, 因此在实践中, 采用标准更新方式

$$\mathbf{a}_i = \mathbf{s}_i + \Delta \mathbf{a}_i$$

就已足够。这里的加法仅表示从 delta action 到绝对动作的转换, 并不表示物理运动的直接叠加。

11.5 F.3 真实世界部署中的实际考虑 (第 2 部分)

表 7: WoVR 真实世界实验中的任务与 embodiment 设置。面板 (a) 报告了任务相关的数据规模, 面板 (b) 总结了不同 embodiment 对应的训练配置。

表 7: 任务相关的数据规模

Embodiment	Task	VLA SFT Demos	WM SFT Rollouts
Franka	Pick banana	25	120
Franka	Pick bread	25	120
Franka	Open drawer	25	120
Agilex	Pick cube	75	180
Agilex	Pick tomato	50	180
Agilex	Fold towel	50	210

(a) 任务相关的数据规模。

表 8: embodiment 相关的训练配置

Embodiment	VLA SFT Steps	WM SFT Steps	Max Episode Steps	RL Epochs
Franka	50k	750k	160	50
Agilex	50k (OpenVLA-OFT), 30k ($\pi_{0.5}$)	1.25M	224	60

(b) **embodiment 相关的训练配置**。相比之下, AgileX Piper 在观测到的 end-effector 状态上存在较明显的噪声与偏差, 因此若直接采用相同的更新规则, 会产生漂移。为缓解这一问题, 我们改为使用累积形式来计算动作:

$$\mathbf{a}_i = \mathbf{s}_0 + \sum_{k=1}^i \Delta \mathbf{a}_k.$$

其中, $\mathbf{s}_0$ 表示给定的初始位置。该形式能够降低系统对噪声反馈的敏感性, 并提升长时程执行的稳定性。

尽管在存在外部扰动 (例如接触) 时, 这一形式可能引入一定偏差, 但这类误差可以由 VLA policy 的视觉反馈闭环加以补偿。

总结。 这些结果表明, 尽管所提出的 delta-space 表达能够跨平台泛化, 但若要实现可靠的真实世界部署, 妥善处理状态估计噪声至关重要。该设计使系统能够在高精度硬件与低成本硬件两类设置下, 都实现稳定的 closed-loop 执行。

11.6 F.4 任务设置

不同任务所使用的 VLA SFT 数据量并不相同, 因为各任务的难度存在差异。对于更困难的任务, 我们分配更多的 demonstration, 以确保监督学习得到的 policy 在进入强化学习之前具备足够的初始性能。用于训练 world model 的数据量也遵循相同原则: 当任务具有更复杂的动力学、更长的时域, 或包含更丰富的交互时, 需要更多 rollout 数据来保证建模的可靠性。

在真实世界实验中, 出于效率考虑, 我们不使用 PACE 进行数据采集。相反, 我们通过对训练过程中不同 step 保存的多个 VLA SFT checkpoint 进行 rollout, 来构建 world-model 的训练集。该策略能够提升轨迹多样性, 并扩大动作空间的覆盖范围, 因为不同训练阶段的 checkpoint 会诱导出不同的行为分布。以 Franka 平台为例, 我们收集来自以下 SFT step 所保存 checkpoint 的 rollout 轨迹: 25k、30k、35k、40k、45k 和 50k。随后, 我们基于真实世界评估选择表现最佳的 checkpoint, 报告其性能, 并将其作为后续 RL 的初始化。